



2025

Servidores y clientes

30 de Octubre.

Diego Vargas Gómez de Requena

Marina Gracia Ontanilla



Índice

Parte Individual Marina.....	3
Instalación de Oracle Database Enterprise Edition 21c en Debian13.....	3
Configuraciones.....	4
Configuración de acceso remoto en Oracle.....	6
Instalación de PostgreSQL bajo Debian 13.....	12
Conexión acceso remoto PostgreSQL.....	12
Instalación MariaDB bajo Debian 13.....	14
Conexión acceso remoto.....	14
Instalación MongoDB.....	15
Configurar acceso remoto.....	15
Instalación Redis.....	21
Guardar datos simples.....	21
Operaciones numéricas.....	21
Guardar datos tipo objeto (Hashes).....	22
Ampliar entrada.....	22
Acceso remoto.....	23
Aplicación Web con MySQL.....	24
Instalación de Flask.....	24
Aplicación.....	24
Login.....	25
Tablas.....	26
Contenido de la tabla.....	27
Parte individual Diego.....	29
Instalación y configuración de Oracle Database EE 21c.....	29
Servidor.....	29
Cliente.....	32
Instalación y configuración de PostgreSQL.....	33
Servidor.....	33
Cliente.....	34
Instalación y configuración de MariaDB.....	34
Servidor.....	34
Cliente.....	35
Instalación y configuración de MongoDB.....	35
Servidor.....	35
Cliente.....	36
Instalación y configuración de Apache Cassandra.....	37
Servidor.....	37
Cliente.....	39
Uso básico de Cassandra.....	39
Creación aplicación web.....	41
Parte grupal.....	43
Neo4j.....	43
Introducción.....	43
Instalación bajo Debian13.....	44
Conexión remota.....	46
Comenzando desde la terminal.....	47
Usando python.....	49
CouchDB.....	51

Instalación.....	52
Creación de base de datos.....	54
Listar bases de datos.....	54
Ver los documentos de la base.....	55
Ver documento específico.....	55
Ver documentos con contenido completo.....	55
Guardar documento desde la terminal.....	56
Usando python.....	56
Conclusión.....	57
Memcached.....	58
Explicación del SGBD.....	58
Instalación y configuración para uso remoto.....	60
Uso básico y explicación del funcionamiento.....	61
Funcionamiento de Memcached.....	61
Uso básico.....	62
Desde Telnet (modo texto).....	62
Desde un lenguaje (Python).....	63
Vídeos.....	64

Parte Individual Marina.

Instalación de Oracle Database Enterprise Edition 21c en Debian13.

Lo primero que haremos será instalar las dependencias, para, posteriormente, poder instalar el software de oracle:

```
[0]marina@debian:~$ sudo apt install rlwrap libaiolt64 libaio-dev unixodbc wget -y
```

Como el paquete libaio ha cambiado de nombre en debian13, oracle al instalarse lo buscará por ese nombre, por lo que voy a crear un enlace simbólico para que lo pueda encontrar:

```
[0]marina@debian:~$ sudo ln -s /usr/lib/x86_64-linux-gnu/libaio.so.1t64 /usr/lib/x86_64-linux-gnu/libaio.so.1
```

Ahora procedemos a descargar oracle enterprise edition, el cual he descargado del siguiente enlace:

[Oracle Database 21c](#)

Una vez hecho esto se nos descarga el .rpm, para pasarlo a .deb necesitamos el paquete 'alien':

```
[0]marina@debian:~$ sudo apt install alien
```

Pasamos de .rmp a .deb:

```
[0]marina@debian:~$ sudo alien --scripts --to-deb /home/marina/Descargas/oracle-database-ee-21c-1.0-1.018.x86_64.rpm
```

Comenzamos la instalación:

```
[0]marina@debian:~$ sudo dpkg -i oracle-database-ee-21c_1.0-2_amd64.deb
```

Configuraciones.

1. A veces, Oracle, necesita resolver el nombre de la máquina con su IP para usar algunas funcionalidades. Gracias a esto Oracle puede usarse de forma remota:

```
marina@debian: ~ 79x22
GNU nano 8.4 /etc/hosts *
127.0.0.1 localhost
127.0.1.1 debian
192.168.12.2 www.marina.org
192.168.1.47 server-oracle
```

Añadimos el nombre de nuestro host con la ip de la máquina.

2. Inicializar y configurar la base de datos de Oracle en nuestro sistema.

```
[0]marina@debian:~$ sudo /etc/init.d/oracledb_ORCLCDB-21c configure
```

Me ha saltado el siguiente error:

'El tamaño de SGA especificado es mayor que el shmmax en el sistema. Esto puede producir un fallo en la creación de la base de datos con el error "ORA-27125: No se ha podido crear el segmento de memoria compartida".

ACCIÓN: Especifique un tamaño de SGA menor o igual que el shmmax del sistema.'

el tamaño de la SGA (System Global Area) que está intentando asignar Oracle es mayor que el valor configurado para shmmax, que es el tamaño máximo de un segmento de memoria compartida en tu sistema. Para ello debemos darle a nuestra máquina virtual mínimo 3GB de RAM y cambiar los parámetros del kernel de la siguiente manera:

```
[0]marina@debian:~$ echo 536870912 | sudo tee /proc/sys/kernel/shmmax
```

3. Dar permisos de administrador de Oracle a mi usuario, para poder ejecutar comandos y administrar la base de datos:

```
[0]marina@debian:~$ sudo usermod -aG dba $USER
```

Ahora tenemos que poner en la .bashrc las variables de entorno, para ello editaremos el .bashrc y le añadimos lo siguiente:

```
echo 'export ORACLE_HOME=/opt/oracle/product/21c/dbhome_1' >> ~/.bashrc
echo 'export ORACLE_SID=ORCLCDB' >> ~/.bashrc
echo 'export NLS_LANG=SPANISH_SPAIN.AL32UTF8' >> ~/.bashrc
echo 'export ORACLE_BASE=/opt/oracle' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=$ORACLE_HOME/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
echo 'export PATH=$ORACLE_HOME/bin:$PATH' >> ~/.bashrc
echo "alias sqlplus='rlwrap sqlplus'" >> ~/.bashrc
```

4. Para iniciar servicio automáticamente.

'sudo contrab -e'

```
@reboot sudo systemctl restart oracledb_ORCLCDB-21c.service
```

Añadimos esa línea al final del archivo.

Configuración de acceso remoto en Oracle.

Para configurar el acceso remoto en Oracle, es fundamental ajustar correctamente los archivos de red ubicados en \$ORACLE_HOME/network/admin. Estos archivos permiten definir cómo se conectarán los clientes a la base de datos y qué equipos tendrán acceso.

```
[0]marina@debian:~$ ls $ORACLE_HOME/network/admin/  
samples shrept.lst
```

samples: Dentro de este directorio hay ejemplos de archivos de configuración para Oracle.

Shrept.lst: Esto es un archivo del proceso de recuperación de Oracle. Se utiliza para rastrear la replicación de registros de cambios en tiempo real.

Dentro del directorio samples encontramos los siguientes archivos:

```
[0]marina@debian:~$ ls $ORACLE_HOME/network/admin/samples/  
listener.ora sqlnet.ora tnsnames.ora
```

listener.ora : Este archivo sirve para configurar el servicio de escucha de Oracle. Contiene información sobre los puntos de conexión y los protocolos que el servidor de Oracle utilizará para aceptar conexiones de clientes.

Encontramos estas líneas en el fichero:

```
LISTENER =  
  (ADDRESS_LIST=  
    (ADDRESS=(PROTOCOL=tcp) (HOST=localhost) (PORT=1521))  
    (ADDRESS=(PROTOCOL=ipc) (KEY=PNPKEY)))
```

ADDRESS_LIST contiene una lista de direcciones donde el listener escuchará conexiones. En mi caso tengo dos direcciones:

- PROTOCOL=tcp: Acepta conexiones TCP/IP.
 - HOST=localhost: Indica que solo escucha en mi máquina local, es decir no se puede acceder desde otras máquinas.
 - PORT=1521: Puerto por el que escucha las conexiones.
- PROTOCOL=ipc: ICP significa Inter-Process Communication, es un método para que procesos en la misma máquina se comuniquen sin usar red.
 - KEY=PNPKEY: Es el identificador de la comunicación ICP. Los clientes locales pueden usarlo para conectarse directamente sin TCP/IP.

Para permitir las conexiones remotas cambiaré HOST=localhost por:

```
. . .  
(ADDRESS=(PROTOCOL=tcp)(HOST=server-oracle)(PORT=1521))  
. . .
```

Esto hace que el listener escuche en la dirección del host. Si quisiéramos que el listener escuche en todas pondríamos 0.0.0.0.

Anteriormente he mencionado una configuración en el /etc/hosts. Este es un archivo que traduce nombres de hosts a direcciones IP:

```
GNU nano 8.4 /etc/hosts *  
127.0.0.1 localhost  
127.0.1.1 debian  
192.168.12.2 www.marina.org  
192.168.1.47 server-oracle
```

Con esto, nos conectaríamos usando ese nombre en vez de la IP, si mi red no tuviese DNS. Si en el tnsnames.ora de los clientes pongo directamente (HOST=192.168.1.47) entonces los clientes se conectarán sin tocar el /etc/hosts.

Ahora copiaremos el archivo en el directorio principal:

```
[1]marina@debian:~$ sudo cp $ORACLE_HOME/network/admin/samples/listener.ora  
$ORACLE_HOME/network/admin/listener.ora  
[0]marina@debian:~$ ls $ORACLE_HOME/network/admin  
listener.ora samples shrept.lst
```

Una vez hecho esto iniciamos servicio de escucha:

```
lsnrctl start
```

Al ejecutarlo me da error de permisos:

```
[0]marina@debian:~$ lsnrctl start  
  
LSNRCTL for Linux: Version 21.0.0.0.0 - Production on 04-OCT-2025 12:05:57  
  
Copyright (c) 1991, 2021, Oracle. All rights reserved.  
  
Iniciando /opt/oracle/product/21c/dbhome_1/bin/tnslsnr: espere...  
  
TNSLSNR for Linux: Version 21.0.0.0.0 - Production  
NL-00280: error al crear el flujo del log  
/opt/oracle/product/21c/dbhome_1/network/log/listener.log  
NL-00278: no se puede abrir el archivo log  
SNL-00016: snlfohd: error al abrir el archivo  
Linux Error: 13: Permission denied  
  
Fallo al iniciar el listener. Consulte los mensajes de error mostrados...
```


Oracle está intentando crear en el archivo de log del listener:

```
NL-00280: error al crear el flujo del log
/opt/oracle/product/21c/dbhome_1/network/log/listener.log
```

Pero mi usuario no tiene permisos de escritura, por lo que voy a cambiar los permisos:

```
[1]marina@debian:~$ sudo chown -R marina:marina
/opt/oracle/product/21c/dbhome_1/network/log
```

Esto permite a mi usuario crear y escribir en el listener.log:

```
[0]marina@debian:~$ lsnrctl start

LSNRCTL for Linux: Version 21.0.0.0.0 - Production on 04-OCT-2025 12:11:39

Copyright (c) 1991, 2021, Oracle. All rights reserved.

Iniciando /opt/oracle/product/21c/dbhome_1/bin/tnslsnr: espere...

TNSLSNR for Linux: Version 21.0.0.0.0 - Production
El archivo de parámetros del sistema es
/opt/oracle/product/21c/dbhome_1/network/admin/listener.ora

Mensajes de log escritos en
/opt/oracle/product/21c/dbhome_1/network/log/listener.log
Recibiendo en: (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=debian)
(PORT=1521)))

Conectándose a (ADDRESS=(PROTOCOL=tcp)(HOST=)(PORT=1521))
ESTADO del LISTENER
-----
Alias                LISTENER
Versión              TNSLSNR for Linux: Version 21.0.0.0.0 - Production
Fecha de Inicio      04-OCT-2025 12:11:39
Tiempo Actividad     0 días 0 hr. 0 min. 0 seg.
Nivel de Rastreo     off
Seguridad            ON: Local OS Authentication
SNMP                 OFF
Parámetros del Listener
/opt/oracle/product/21c/dbhome_1/network/admin/listener.ora
Log del Listener
/opt/oracle/product/21c/dbhome_1/network/log/listener.log
Recibiendo Resumen de Puntos Finales...
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=debian)(PORT=1521)))
El listener no soporta ningún servicio
El comando ha terminado correctamente
```

Para que nos encuentre el servicio debemos añadir las variables de oracle en el .bashrc como hicimos anteriormente.

Para comprobar conectividad lo haré desde el pc de mi compañero Diego Vargas:

```
sqlplus usuario/password@//server-oracle/ORCLCDB
```

```
> whoami
diego
> sqlplus marina/marina@//172.22.0.27/ORCLCDB

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Oct 6 09:39:27 2025
Version 21.19.0.0.0

Copyright (c) 1982, 2022, Oracle. All rights reserved.

Hora de Ultima Conexion Correcta: Lun Oct 06 2025 09:38:56 +02:00

Conectado a:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> SELECT table_name FROM user_tables;

TABLE_NAME
-----
ALUMNOS
PROFESORES
CURSOS
MATRICULAS
OPOSICIONES
CURSO_OPOSICION
MANUALES
CURSO_MANUAL

8 filas seleccionadas.
```

Tras instalar oracle me encuentro con el problema de que al iniciar la máquina tengo que arrancar el servicio manualmente, entrando dentro de oracle y haciendo 'startup'. Al hacer `sudo systemctl enable oracle`:

```
[0]marina@debian:~$ sudo systemctl enable oracledb_ORCLCDB-21c.service
oracledb_ORCLCDB-21c.service is not a native service, redirecting to
systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable oracledb_ORCLCDB-21c
update-rc.d: error: oracledb_ORCLCDB-21c Default-Start contains no
runlevels, aborting.
```

Esto nos indica que el script de inicio de Oracle no está completamente configurado para systemd. Systemctl enable intenta redirigirlo a 'systemd-sysv-install' pero falla porque no contiene los 'runlevels' necesarios. Es decir, que mi script no puede habilitarse automáticamente al inicio tal y como está.

Voy a crear un servicio para que se inicie automáticamente al arrancar:

```
[1]marina@debian:~$ sudo nano /etc/systemd/system/oracledb.service
[Unit]
#información general del servicio
Description=Oracle Database 21c
After=network.target
#el after es para que se inicie después de que la red esté disponible.

[Service]
#como se ejecuta el servicio
Type=forking
User=oracle
Group=oinstall
Environment="ORACLE_HOME=/opt/oracle/product/21c/dbhome_1"
Environment="ORACLE_SID=ORCLCDB"
Environment="PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:>
ExecStart=/opt/oracle/product/21c/dbhome_1/bin/dbstart $ORACLE_HOME
ExecStop=/opt/oracle/product/21c/dbhome_1/bin/dbshut $ORACLE_HOME
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
#esto es para decirle a systemd que arranque automaticamente en el
#nivel de ejecución normal del sistema
```

Para que systemd reconozca el nuevo servicio:

```
[0]marina@debian:~$ sudo systemctl daemon-reload
```

Habilitar servicio:

```
[0]marina@debian:~$ sudo systemctl enable oracledb.service
Created symlink
'/etc/systemd/system/multi-user.target.wants/oracledb.service' →
'/etc/systemd/system/oracledb.service'.
```

Arrancamos servicio:

```
[0]marina@debian:~$ sudo systemctl enable oracledb.service
Created symlink
'/etc/systemd/system/multi-user.target.wants/oracledb.service' →
'/etc/systemd/system/oracledb.service'.
[0]marina@debian:~$ sudo systemctl start oracledb.service
```

Después de crear el servicio accedemos al fichero de oracle /etc/oratab. Es un fichero de oracle que indica que bases de datos levantar automáticamente y dónde está instalado todo. Añadimos lo siguiente:

```
ORCLCDB:/opt/oracle/product/21c/dbhome_1:Y
```

- ORCLCDB: Es el identificador de la base de datos (\$ORACLE_SID).
- /opt/oracle/product/21c/dbhome_1: Ruta donde está instalado oracle.
- Y: Indica si dbstart debe iniciar esta base al arrancar (Y=YES).

Ya estaría solucionado el error.

Instalación de PostgreSQL bajo Debian 13.

Antes de empezar, como siempre, actualizamos los repositorios e instalamos actualizaciones.

```
debian@debian:~$ sudo apt update && sudo apt upgrade -y
```

Comenzamos con la instalación.

```
debian@debian:~$ sudo apt install postgresql postgresql-contrib
```

Revisamos el estado del servicio:

```
debian@debian:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled;
   prese>
   Active: active (exited) since Sat 2025-10-04 13:03:10 CEST; 22s ago
   Invocation: aba5169d51c34bef9bc5a9377ffaae79
   Main PID: 6082 (code=exited, status=0/SUCCESS)
```

Conexión acceso remoto PostgreSQL.

Para que acepte conexiones TCP/IP modificamos el siguiente archivo:

```
GNU nano 8.4      /etc/postgresql/17/main/postgresql.conf *
listen_addresses = 'localhost'          # what IP address(es) to listen on;
----- lo ponemos:
listen_addresses = '192.168.122.182'    # what IP address(es) to
listen on;
```

Este es el archivo principal de configuración de PostgreSQL. Le dice al servidor como debe iniciar, donde escuchar conexiones, que puerto usar, etc. Si quisiéramos escuchar en todas las interfaces disponibles pondríamos '*'.

El siguiente archivo que debemos configurar es el pg_hba.conf, donde se configura quien puede conectarse al servidor y como se autentica:

```
GNU nano 8.4      /etc/postgresql/17/main/pg_hba.conf *
# IPv4 local connections:
host      all             all             127.0.0.1/32             scram-
sha-256
host      all             all             192.168.122.0/24         scram-
sha-256
```

El método de autenticación se establece con 'scram-sha-256'. Se usa para verificar la identidad de un usuario sin enviar contraseña en texto plano. De esta forma, la contraseña original nunca viaja por la red, por lo que aunque se capture el tráfico nunca se podrá reconstruir la contraseña.

Hay diferentes métodos de autenticación en PostgreSQL, pero el más seguro es este.

Reiniciamos el servicio:

```
debian@debian:~$ sudo systemctl restart postgresql
```

Ahora comprobamos que el servidor escucha en la red:

```
debian@debian:~$ sudo ss -tulnp | grep 5432
tcp    LISTEN 0      200      192.168.122.182:5432
0.0.0.0:*    users:(("postgres",pid=1214,fd=6))
```

Ahora comprobamos conexión desde otro pc de la red, para ello lo primero será crear un usuario:

```
postgres=# CREATE USER marina WITH PASSWORD 'marina';
CREATE ROLE
postgres=# ALTER USER marina CREATEDB;
ALTER ROLE
postgres=# CREATE DATABASE prueba OWNER marina;
CREATE DATABASE
postgres=# GRANT ALL PRIVILEGES ON DATABASE prueba TO marina;
GRANT
```

Comprobamos conexión:

```
[0]marina@debian:~$ psql -h 192.168.122.182 -U marina -d prueba
Contraseña para usuario marina:
psql (17.6 (Debian 17.6-0+deb13u1))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384,
compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.

prueba=>
```

Instalación MariaDB bajo Debian 13.

Actualizamos repositorios:

```
debian@debian:~$ sudo apt update && sudo apt upgrade -y
```

E instalamos, habilitamos e iniciamos el servidor:

```
debian@debian:~$ sudo apt install mariadb-server
debian@debian:~$ sudo systemctl enable mariadb
debian@debian:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.3 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; pres>
   Active: active (running) since Sun 2025-10-05 13:24:13 CEST; 1min 20
```

Creamos un nuevo usuario:

```
debian@debian:~$ sudo mysql -u root -p
MariaDB [(none)]> CREATE USER 'marina'@'%' IDENTIFIED BY 'marina';
Query OK, 0 rows affected (0,005 sec)
--creamos BBDD
MariaDB [(none)]> CREATE DATABASE prueba;
```

Conexión acceso remoto.

Configuramos el archivo:

```
GNU nano 8.4      /etc/mysql/mariadb.conf.d/50-server.cnf
bind-address      = 192.168.122.182
```

Restauramos servicio:

```
debian@debian:~$ sudo systemctl restart mariadb
```

Para permitir que el usuario creado en la base de datos tenga acceso desde cualquier PC de mi red local:

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON prueba.* TO
'marina'@'192.168.122.%' IDENTIFIED BY 'marina';
```

Comprobamos conexión:

```
[0]marina@debian:~$ mysql -h 192.168.122.182 -u marina -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
. . .
MariaDB [(none)]> use prueba
Database changed
MariaDB [prueba]>
```

Instalación MongoDB.

Lo primero que haremos será descargar el paquete desde la página oficial de mongo. Todavía no hay una versión específica para Debian 13, por lo que instalaremos la versión de bookworm:

```
[0]marina@debian:~$ wget
https://repo.mongodb.org/apt/debian/dists/bookworm/mongodb-org/7.0/main/
binary-amd64/mongodb-org-server_7.0.22_amd64.deb
```

A continuación lo instalamos:

```
[0]marina@debian:~$ sudo dpkg -i mongodb-org-
server_7.0.22_amd64.debActivamos el servicio:
[0]marina@debian:~$ sudo systemctl start mongod.service
[0]marina@debian:~$ sudo systemctl enable mongod.service
```

Configurar acceso remoto.

Accedemos al fichero de configuración /etc/mongod.conf:

```
# network interfaces
net:
  port: 27017
  bindIp: 127.0.0.1,192.168.1.47
```

Reiniciamos mongo:

```
[0]marina@debian:~$ sudo systemctl restart mongod.service
```

Y comprobamos que escucha por el puerto 27017:

```
[0]marina@debian:~$ sudo ss -tuln | grep 27017
tcp    LISTEN 0      4096      127.0.0.1:27017      0.0.0.0:*
tcp    LISTEN 0      4096     192.168.1.47:27017   0.0.0.0:*
```

A continuación, instalaré el cliente de mongo. Para ello importo la clave oficial de MongoDB:

```
debian@marina:~$ wget -qO -
https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg --dearmor -o
/usr/share/keyrings/mongodb-server-7.0.gpg
```


Añado el repositorio APT:

```
debian@marina:~$ echo "deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] https://repo.mongodb.org/apt/debian bookworm/mongodb-org/7.0 main" | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
deb [ signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] https://repo.mongodb.org/apt/debian bookworm/mongodb-org/7.0 main
```

Actualizo e instalo solo el cliente:

```
[0]marina@debian:~$ sudo apt update
debian@marina:~$ sudo apt install mongodb-mongosh -y
```

Ahora crearé un usuario con mongosh:

```
[0]marina@debian:~$ mongosh
test> use admin
switched to db admin
admin>prueba> db.createUser({ user: "marina", pwd: "marina", roles: [{ role: "readWrite", db: "prueba" }] })
{ ok: 1 }
```

Habilitamos autenticación en mongod.conf:

```
security:
  authorization: enabled
```

Comprobamos conexión remota:

```
debian@marina:~$ mongosh "mongodb://marina:marina@192.168.1.47:27017/prueba"
Current Mongosh Log ID: 68e3cb9badbc38fa7dce5f46
Connecting to: mongodb://<credentials>@192.168.1.47:27017/prueba?directConnection=true&appName=mongosh+2.5.8
Using MongoDB: 7.0.22
Using Mongosh: 2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

prueba>
```

Si configuramos el /etc/hosts de la siguiente manera:

```
192.168.1.47    servidor.mongo
```

Podremos acceder usando el nombre que establezcamos:

```
debian@marina:~$ mongosh
"mongodb://marina:marina@servidor.mongo:27017/prueba"
Current Mongosh Log ID: 68e3cbceabb51fc4f2ce5f46
Connecting to: mongodb://<credentials>@servidor.mongo:27017/prueba?
directConnection=true&appName=mongosh+2.5.8
Using MongoDB: 7.0.22
Using Mongosh: 2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

prueba> exit
```

Para **crear una colección** podemos hacerlo de la siguiente forma:

```
prueba> db.clientes.insert({
... "name" : "antonio",
... "age" : 22,
... "location" : "cadiz"
... })
DeprecationWarning: Collection.insert() is deprecated. Use insertOne,
insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('68e3d24393e345a2dfce5f47') }
}
```

Para **crear una colección sin insertar datos**:

```
prueba> db.createCollection("productos",
... {capped: true, size: 500, max: 15})
```

- capped: Indica que la colección tiene un tamaño máximo definido. Cuando se alcance el límite, los documentos más antiguos se eliminan automáticamente.
- Size: Establece el tamaño de la colección en bytes.
- Max: Define el número máximo de documentos que la colección puede contener.

Para **insertar datos** nos encontramos con tres métodos:

- insertOne(): Para insertar un solo documento.

```
prueba> db.productos.insertOne(
... {
... "name": "raton",
... "precio": 22
... })
{
  acknowledged: true,
  insertedId: ObjectId('68e3d4c993e345a2dfce5f48')
}
```

- `InsertMany()`: Para insertar mas de un documento.

```
prueba> db.productos.insertMany([{"name": "alfombrilla", "precio": 12},
... {"name": "teclado", "precio": 35},
... {"name": "silla", "precio": 97}
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68e3d74c93e345a2dfce5f49'),
    '1': ObjectId('68e3d74c93e345a2dfce5f4a'),
    '2': ObjectId('68e3d74c93e345a2dfce5f4b')
  }
}
prueba>
```

- `Insert()`: Para insertar tantos documentos como se desee.

```
prueba> db.clientes.insert([
... {"name": "marina", "age": 24},
... {"name": "elena", "age": 24}
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68e3d7a593e345a2dfce5f4c'),
    '1': ObjectId('68e3d7a593e345a2dfce5f4d')
  }
}
prueba>
```

Para **consultar datos** podemos usar la función `'db.collection.find'` y si quisiéramos verlo en un formato mas limpio usaríamos `'db.collection.find().pretty()'`:

```
prueba> db.clientes.find().pretty()
[
  {
    _id: ObjectId('68e3d24393e345a2dfce5f47'),
    name: 'antonio',
    age: 22,
    location: 'cadiz'
  },
  {
    _id: ObjectId('68e3d7a593e345a2dfce5f4c'),
    name: 'marina',
    age: 24
  },
  { _id: ObjectId('68e3d7a593e345a2dfce5f4d'), name: 'elena', age: 24 }
]
prueba>
```

Vemos como aparece un id, esto sucede porque mongo añade automáticamente un id para identificar de forma única cada documento. Si no queremos que se muestre el id:

```
prueba> db.clientes.find({}, { _id: 0 }).pretty()
[
  { name: 'antonio', age: 22, location: 'cadiz' },
  { name: 'marina', age: 24 },
  { name: 'elena', age: 24 }
]
```

Si quisiéramos **filtrar los datos**, podemos especificarlo de la siguiente manera:

```
prueba> db.clientes.find({name: "marina"})
[
  {
    _id: ObjectId('68e3d7a593e345a2dfce5f4c'),
    name: 'marina',
    age: 24
  }
]
```

Si quisiéramos mostrar solo personas menores de 25 años:

```
prueba> db.clientes.find({age : {$lt : 25}}, { _id: 0 })
[
  { name: 'antonio', age: 22, location: 'cadiz' },
  { name: 'marina', age: 24 },
  { name: 'elena', age: 24 }
]
```

Para **actualizar**, por ejemplo, la edad de uno de mis clientes:

```
prueba> db.clientes.updateOne({_id: ObjectId('68e3d24393e345a2dfce5f47')},
{$set: {age:23}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
prueba> db.clientes.find({age : {$lt : 25}}, { _id: 0 })
[
  { name: 'antonio', age: 23, location: 'cadiz' },
  { name: 'marina', age: 24 },
  { name: 'elena', age: 24 }
]
prueba>
```

También podemos utilizar `updateMany`. Vamos a hacer que todos los clientes menores de 25, tengan 25:

```
prueba> db.clientes.updateMany({age: {$lt: 25}},{$set: {age: 25}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
prueba> db.clientes.find({age : 25},{ _id: 0})
[
  { name: 'antonio', age: 25, location: 'cadiz' },
  { name: 'marina', age: 25 },
  { name: 'elena', age: 25 }
]
prueba>
```

Para eliminar documentos tenemos varias funciones:

- `deleteOne()`: Elimina el primer documento que coincide con lo que se busca.

```
prueba> db.clientes.deleteOne({name: "antonio"})
{ acknowledged: true, deletedCount: 1 }
prueba> db.clientes.find({}, { _id: 0})
[ { name: 'marina', age: 25 }, { name: 'elena', age: 25 } ]
prueba>
```

- `deleteMany()`: Elimina todos los documentos que coincida con el filtro:

```
prueba> db.clientes.deleteMany({age: 25})
{ acknowledged: true, deletedCount: 2 }
prueba> db.clientes.find({}, { _id: 0})
```

Para **eliminar una colección** podemos usar:

```
prueba> db.clientes.drop()
true
```

Instalación Redis.

Redis es una base de datos basado en el almacenamiento Redis es un sistema de almacenamiento de datos en memoria (in-memory) que se utiliza principalmente como base de datos, caché y broker de mensajes.

```
[0]marina@debian:~$ sudo apt install redis-server
```

Comprobamos servicio:

```
[0]marina@debian:~$ sudo systemctl status redis.service
● redis-server.service - Advanced key-value store
   Loaded: loaded (/usr/lib/systemd/system/redis-server.service; enabled;>
   Active: active (running) since Sun 2025-10-05 13:48:56 CEST; 3min 51s
   CGroup: /system.slice/redis-server.service
           └─13825 "/usr/bin/redis-server 127.0.0.1:6379"
```

Una vez instalado, para abrir la terminal interactiva de redis:

```
[0]marina@debian:~$ redis-cli
127.0.0.1:6379>
```

Guardar datos simples.

Los Strings son la forma más simple de almacenar información:

```
127.0.0.1:6379> set nombre "Antonio"
OK                                     #guarda valor "Antonio" en la clave "nombre"
127.0.0.1:6379> get nombre
"Antonio"                             #muestra el valor "nombre"
127.0.0.1:6379> del nombre
(integer) 1                           #elimina la clave "nombre"
```

Operaciones numéricas.

```
127.0.0.1:6379> set contador 5
OK
127.0.0.1:6379> INCR contador
(integer) 6                           #incrementa en 1
127.0.0.1:6379> DECR contador
(integer) 5                           #decrementa en 1
127.0.0.1:6379>
```

Guardar datos tipo objeto (Hashes)

Los hashes son entradas individuales compuestas de varios valores, de manera similar a los sets y las listas, pero en los que cada valor va acompañado de una clave, formando así los llamados pares clave-valor o key-value

```
127.0.0.1:6379> HSET usuario:1 nombre "Antonio" edad 22
(integer) 2
127.0.0.1:6379> HGET usuario:1 nombre
"Antonio"
127.0.0.1:6379> HGETALL usuario:1
1) "nombre"
2) "Antonio"
3) "edad"
4) "22"
127.0.0.1:6379>
```

Esto es ideal para usuarios, productos y configuraciones.

Ampliar entrada.

Una vez se ha creado una entrada, esta se puede ampliar: el comando append añade otro valor al que ya existía.

```
127.0.0.1:6379> set nombre "Marina"
OK
127.0.0.1:6379> APPEND nombre " Gracia"
(integer) 13
127.0.0.1:6379> GET nombre
"Marina Gracia"
```

Como se puede ver, al solicitar los valores correspondientes, aparecen los nuevos elementos simplemente tras los que ya estaban. Si la entrada en cuestión aún no existe, append realiza entonces la misma función que set.

Además, también se puede cambiar el nombre de las entradas usando el comando rename.

```
127.0.0.1:6379> rename nombre cliente
OK
127.0.0.1:6379> get cliente
"Marina Gracia"
```

Acceso remoto.

Tendremos que modificar su configuración de red:

```
#Cambiaremos esta línea:
bind 127.0.0.1 ::1
#por esta:
bind 0.0.0.0                # listens on loopback IPv4 and IPv6
```

Esto permite a Redis escuchar por todas las interfaces de red. Establecemos una contraseña. Para ello buscamos la siguiente línea:

```
# requirepass foobared
```

Y la cambiamos por:

```
requirepass contraseña_aqui
```

Abrimos el puerto de escucha de redis:

```
[0]marina@debian:~$ sudo ufw allow 6379/tcp
```

Y reiniciamos:

```
[0]marina@debian:~$ sudo systemctl restart redis
```

Comprobamos que está escuchando:

```
[0]marina@debian:~$ sudo ss -tlnp | grep 6379
LISTEN 0      511          127.0.0.1:6379      0.0.0.0:*    users:(("redis-
server",pid=8612,fd=8))
LISTEN 0      511
```

Comprobamos conexión desde un cliente de una VM, al que anteriormente le he instalado redis-cli con el comando:

```
sudo apt install redis-tools -y
```

Y nos conectamos:

```
debian@debian:~$ redis-cli -h 192.168.1.47 -p 6379 -a CONTRASEÑA
Warning: Using a password with '-a' or '-u' option on the command line
interface may not be safe.
192.168.1.47:6379> ping
PONG
```


Aplicación Web con MySQL.

Para mi aplicación web voy a utilizar Flask, un framework de python, que permite crear aplicaciones web dinámicas.

Instalación de Flask.

Instalamos python3:

```
[1]marina@debian:~$ sudo apt install python3
```

Para aislar las dependencias del proyecto, creamos un entorno virtual y en el instalamos flask:

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/apWeb$ python3 -m venv env
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/apWeb$ source env/bin/activate
(env) [0]marina@debian:~/Escritorio/BBDD/tema1/practicas/apWeb$ pip install Flask
```

Aplicación.

Creamos un archivo llamado app.py:

```
(env) [0]marina@debian:~/Escritorio/BBDD/tema1/practicas/apWeb$ export FLASK_APP=app.py
```

En primer lugar, importamos las librerías que vamos a necesitar para la aplicación.

```
from flask import Flask, request, render_template, redirect, url_for, session
from dotenv import load_dotenv
import pymysql, os
```

- request: Nos permite acceder a los datos enviados por el usuario con el método POST o GET.
- render_template: Muestra los archivos HTML que se encuentran en templates.
- Redirect y url_for: Es lo que permite movernos entre rutas de forma dinámica.
- Session: Para almacenar información del usuario durante la sesión.
- load_dotenv: Es para cargar la clave del archivo .clave.env.
- Pymysql: es la librería que nos permite conectar Python con MariaDB.
- Os: Se utiliza para acceder a las variables del sistema.

Para cargar la contraseña:

```
load_dotenv('clave.env') → Carga la clave desde el archivo clave.env y así  
no ponemos la clave en el código.  
app = Flask(__name__) → Se crea la aplicación  
app.secret_key=os.getenv('SECRET_KEY') → protege las sesiones
```

Esto evita incluir la clave directamente en el código.

Login

```
@app.route('/', methods=['GET', 'POST'])  
def login():  
    if request.method == 'POST':  
        host = request.form['host']  
        user = request.form['usuario']  
        password = request.form['clave']  
        database = request.form['basedatos']  
  
        try:  
            conexion = pymysql.connect(  
                host=host,  
                user=user,  
                password=password,  
                database=database  
            )  
            conexion.close()  
  
            # Guardamos los datos en sesión  
            session['host'] = host  
            session['user'] = user  
            session['password'] = password  
            session['database'] = database  
  
            # Redirige a la página de tablas  
            return redirect(url_for('mostrar_tablas'))  
        except Exception as e:  
            return f"Error al conectar con MySQL: {e}"  
  
    return render_template('login.html')
```

Si el método HTTP es POST, se obtiene los datos del formulario mediante request.form. Se intenta establecer una conexión con MySQL utilizando pymysql.connect().

Si la conexión es correcta, los datos se guardan en la sesión y el usuario pasa a la siguiente ruta /tablas. Si ocurre un error se muestra un mensaje de fallo.

Si el método es GET, se muestra la página de login.

El HTML de formulario sería:

```
. . .
<h1>Conectar a MySQL</h1>
<form method="POST">
    Host: <input type="text" name="host" value="localhost"><br>
    Usuario: <input type="text" name="usuario"><br>
    Clave: <input type="password" name="clave"><br>
    Base de datos: <input type="text" name="basedatos"><br>
    <button type="submit">Conectar</button>
</form>
. . .
```

Tablas.

Una vez se establece la conexión con éxito, se redirige al usuario a la siguiente ruta, donde vamos a mostrar las tablas sobre las que ese usuario tiene permisos:

```
@app.route('/tablas')
def mostrar_tablas():
    if 'user' not in session:
        return redirect(url_for('login'))

    conexion = pymysql.connect(
        host=session['host'],
        user=session['user'],
        password=session['password'],
        database=session['database']
    )
    cursor = conexion.cursor()
    cursor.execute("SHOW TABLES;")
    tablas = [t[0] for t in cursor.fetchall()]
    conexion.close()

    return render_template('tablas.html', tablas=tablas)
```

Aquí comprobamos si el usuario ha iniciado sesión, se realiza un show tables para mostrar las tablas y el resultado se envía al HTML:

```
<body>
<h1>Tablas en la base de datos</h1>
<ul>
{% for t in tablas %}
<li><a href="{{ url_for('mostrar_registros', nombre=t) }}">{{ t }}</a></li>
{% endfor %}
</ul>
</body>
```

Mostramos una lista con los enlaces a cada tabla. Cada enlace llama a la ruta /tabla/nombre donde se mostrará su contenido.

Contenido de la tabla.

```
@app.route('/tabla/<nombre>')
def mostrar_registros(nombre):
    if 'user' not in session:
        return redirect(url_for('login'))

    conexion = pymysql.connect(
        host=session['host'],
        user=session['user'],
        password=session['password'],
        database=session['database']
    )
    cursor = conexion.cursor()
    cursor.execute(f"SELECT * FROM {nombre};")
    registros = cursor.fetchall()
    columnas = [desc[0] for desc in cursor.description]
    conexion.close()

    return render_template('registros.html', nombre=nombre,
                           columnas=columnas, registros=registros)
```

Con `cursor.description` sacamos los nombres de las columnas. Con `fetchall` sacamos el contenido completo de la tabla y se renderiza la plantilla `registros.html`:

```
<h1>Registros de {{ nombre }}</h1>
<table border="1">
<tr>
{% for c in columnas %}
<th>{{ c }}</th>
{% endfor %}
</tr>
{% for fila in registros %}
<tr>
{% for celda in fila %}
<td>{{ celda }}</td>
{% endfor %}
</tr>
{% endfor %}
</table>
</body>
</html>
```

Generamos dinámicamente una tabla HTML donde cada columna y celda se corresponde con los datos obtenidos de MariaDB.

Y así se vería la aplicación web:

Conectar a MySQL

Host:

Usuario:

Clave:

Base de datos:

Tablas en la base de datos

- [alquileres](#)
- [clientes](#)
- [vehiculos](#)

Registros de la tabla clientes

dni	nombre	nacionalidad	fecha_nacimiento	direccion
11223344C	Carlos Martinez	Mexicana	1992-12-05	Calle Luna 45, Sevilla
12345678A	Juan Perez	Española	1990-04-20	Calle Mayor 1, Madrid
12345678X	Juan Perez	Española	1985-06-15	Calle Falsa 123
22334455G	Pedro Ruiz	Española	1983-01-05	Calle Nueva 12, Madrid
33445566E	Luis Fernandez	Chilena	1991-06-25	Calle Verdes 18, Zaragoza
55667788D	Ana Lopez	Argentina	1988-03-30	Paseo Maritimo 67, Valencia
66778899H	Sofia Morales	Española	1994-08-17	Calle Castilla 45, Bilbao
77889900F	Laura Torres	Peruana	1987-11-05	Calle Pinos 90, Alicante
87654321B	Maria Garcia	Española	1985-07-15	Avenida Sol 23, Barcelona

Si el usuario logueado tiene otros permisos cambia el contenido, solo muestra el contenido sobre el que el usuario tiene permisos:

Tablas en la base de datos • [clientes](#)

Enlace al código en [GitHub](#).

Parte individual Diego.

Instalación y configuración de Oracle Database EE 21c

Servidor

Antes de instalar nada voy a explicar que Oracle Database no da paquetes de instalación para sistemas como Debian que usan .deb, por eso el paquete .rpm que si ofrece Oracle Database lo voy a convertir con alien a .deb. Ese paquete .deb lo he subido a un servidor de archivos que tengo para poder instalarlo de forma más rápida posteriormente.

Una vez aclarado esto voy a empezar con la instalación de Oracle Database con los siguientes comandos:

```
sudo apt install libaio1t64 libaio-dev unixodbc rlwrap wget -y
```

Antes para instalar Oracle Database en Debian 12 se necesitaba el paquete libaio1 pero en Debian 13 ese paquete ha cambiado de nombre (libaio1t64).

Como Oracle Database busca el fichero /usr/lib/x86_64-linux-gnu/libaio.so.1 pero ese fichero en Debian 13 ahora se llama /usr/lib/x86_64-linux-gnu/libaio.so.1t64 vamos a crear un enlace simbólico del archivo actual que se llame como el antiguo para que Oracle Database lo encuentre:

```
sudo ln -s /usr/lib/x86_64-linux-gnu/libaio.so.1t64 /usr/lib/x86_64-linux-gnu/libaio.so.1
```

Ahora empezaré con la descargar del archivo .deb y la instalación del mismo:

```
wget https://files.diegovargas.es/deb/oracle-database-ee-21c\_1.0-2\_amd64.deb  
sudo dpkg -i oracle-database-ee-21c_1.0-2_amd64.deb
```

Ya casi finalizando la instalación realizaré unos comandos para la configuración del servidor Oracle Database:

```
echo "${hostname -I | awk '{print $1}')} ${hostname}" | sudo tee -a  
/etc/hosts  
sudo /etc/init.d/oracledb_ORCLCDB-21c configure  
sudo usermod -aG dba $USER
```

El primer comando es útil, ya que Oracle Database necesita resolver el nombre del host con una IP válida. Si no se configurara, la instalación fallaría y habría problemas al arrancar el listener.

Ahora como última configuración necesaria será asignar los alias para poder usar Oracle Database.

```
echo 'export ORACLE_HOME=/opt/oracle/product/21c/dbhome_1' >> ~/.bashrc
echo 'export ORACLE_SID=ORCLCDB' >> ~/.bashrc
echo 'export NLS_LANG=SPANISH_SPAIN.AL32UTF8' >> ~/.bashrc
echo 'export ORACLE_BASE=/opt/oracle' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=$ORACLE_HOME/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
echo 'export PATH=$ORACLE_HOME/bin:$PATH' >> ~/.bashrc
echo "alias sqlplus='rlwrap sqlplus'" >> ~/.bashrc
source ~/.bashrc
```

Lo siguiente que haré será crear un usuario para poder posteriormente conectarme remotamente:

```
sqlplus / as sysdba
ALTER SESSION SET "_ORACLE_SCRIPT"=true;
CREATE USER usuario IDENTIFIED BY usuario;
GRANT ALL PRIVILEGES TO usuario;
```

Para que el servicio se inicie automáticamente voy a meter la siguiente entrada como tarea cron:

```
sudo crontab -e
@reboot sudo systemctl restart oracledb_ORCLCDB-21c.service
```

Ahora pasaré a configurar el servidor para que escuche las peticiones de la red y no solo del localhost.

Para lograrlo tendré que modificar el fichero de configuración **/opt/oracle/homes/OraDBHome21cEE/network/admin/listener.ora**:

```
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.122.109)(PORT = 1521))
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
  )
```

Como se ve en la captura se debe cambiar lo que está después de HOST por la IP del equipo servidor, esto indica cualquier dirección IP de la red.

Ahora se hará lo mismo en el archivo **/opt/oracle/homes/OraDBHome21cEE/network/admin/tnsnames.ora**:

```
ORCLCDB =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.122.109) (PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = ORCLCDB)
    )
  )
```

Nota: Otra opción en vez de poner la IP del servidor en cada archivo es dejar el nombre del equipo que es el que se ha configurado en los DNS estáticos anteriormente con el comando `echo "\$(hostname -I | awk '{print \$1}')" \$(hostname)" | sudo tee -a /etc/hosts`.

Ahora para aplicar los cambios será necesario reiniciar el servicio listener con los siguientes comandos:

```
sudo cp /opt/oracle/homes/OraDBHome21cEE/network/admin/listener.ora
/opt/oracle/product/21c/dbhome_1/network/admin/
sudo cp /opt/oracle/homes/OraDBHome21cEE/network/admin/tnsnames.ora
/opt/oracle/product/21c/dbhome_1/network/admin/
sudo chown oracle:oinstall
/opt/oracle/product/21c/dbhome_1/network/admin/listener.ora
sudo chown oracle:oinstall
/opt/oracle/product/21c/dbhome_1/network/admin/tnsnames.ora
sudo /opt/oracle/product/21c/dbhome_1/bin/lsnrctl stop
sudo /opt/oracle/product/21c/dbhome_1/bin/lsnrctl start
```


Ciente

A continuación pasaré a instalar el cliente para poder conectarme remotamente a Oracle Database.

Lo primero será instalar los siguientes paquetes:

```
sudo apt install libaio1t64 libaio-dev rlwrap wget p7zip-full -y
```

Con el paquete libaio1t64 pasa lo mismo que para instalar el servidor Oracle Database, hay que crear el link para que Debian 13 lo encuentre:

```
sudo ln -s /usr/lib/x86_64-linux-gnu/libaio.so.1t64 /usr/lib/x86_64-linux-gnu/libaio.so.1
A continuación pasaré con la instalación del cliente, para ello será necesario basic y sqlplus:
mkdir ~/oracle
cd ~/oracle
wget https://download.oracle.com/otn_software/linux/instantclient/2119000/instantclient-basic-linux.x64-21.19.0.0.0dbru.zip
wget https://download.oracle.com/otn_software/linux/instantclient/2119000/instantclient-sqlplus-linux.x64-21.19.0.0.0dbru.zip
7z x instantclient-basic-linux.x64-21.19.0.0.0dbru.zip
7z x instantclient-sqlplus-linux.x64-21.19.0.0.0dbru.zip
```

Por último para que el sistema pueda ejecutar los binarios será necesario crear las variables:

```
echo 'export ORACLE_HOME=$HOME/oracle/instantclient_21_19' >> ~/.bashrc
echo 'export PATH=$ORACLE_HOME:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=$ORACLE_HOME:$LD_LIBRARY_PATH' >> ~/.bashrc
echo "alias sqlplus='rlwrap sqlplus'" >> ~/.bashrc
source ~/.bashrc
```

Ahora ya podré conectarme remotamente al servidor:

```
sqlplus usuario/usuario@//192.168.122.109:1521/ORCLCDB
```

Instalación y configuración de PostgreSQL

Servidor

Lo primero que voy a realizar es la instalación del servidor, como estoy usando Debian 13 lo haré con el siguiente comando:

```
sudo apt install postgresql -y
```

Con esto el servidor esta instalado, ahora lo configuraré para que escuche peticiones de la red local para que se puedan conectar los equipos.

Como yo estoy instalando el servidor en una máquina virtual de QEMU/KVM la dirección de red que voy a usar para las conexiones es la NAT por defecto que tiene QEMU/KVM que es la 192.168.122.0/24.

Modificaré el fichero de configuración **/etc/postgresql/17/main/pg_hba.conf**:

```
# "local" is for Unix domain socket connections only
local    all             all                                scram-
sha-256
# IPv4 local connections:
host     all             all                                192.168.122.0/24    scram-
sha-256
```

Esta configuración lo que permite es poder validarte con el cifrado scram-sha-256 desde el mismo servidor y poder usar el mismo cifrado para validarte desde una dirección de la red 192.168.122.0/24.

También será necesario modificar el fichero **/etc/postgresql/17/main/postgresql.conf**:

```
listen_addresses = '*'                                # what IP address(es) to listen on;
```

Esto para que escuche en todas las interfaces pero solamente validará las de la red 192.168.122.0/24.

Será necesario reiniciar el servicio:

```
sudo systemctl restart postgresql
```

Por último en la parte del servidor crearé un usuario y una base de datos para conectarme posteriormente desde un cliente, para ello usaré los siguientes comandos:

```
sudo -u postgres psql
\password postgres
CREATE USER usuario WITH PASSWORD 'usuario';
CREATE DATABASE db OWNER usuario;
GRANT ALL PRIVILEGES ON DATABASE db TO usuario;
```

Cliente

Lo primero será instalar el cliente, lo haré con el siguiente comando:

```
sudo apt install postgresql-client -y
```

Ahora para poder conectarme a la base de datos creada anteriormente usaré el siguiente comando:

```
psql -h 192.168.122.218 -U usuario -d db
```

Instalación y configuración de MariaDB

Servidor

Empezaré igual que PostgreSQL, instalaré el paquete del servidor MariaDB:

```
sudo apt install mariadb-server -y
```

Ahora configuraré al igual que en PostgreSQL los ficheros para que MariaDB escuche peticiones remotas de la red.

El fichero de configuración es **/etc/mysql/mariadb.conf.d/50-server.cnf** y tenemos que modificar lo siguiente:

```
bind-address          = 0.0.0.0
```

Ahora MariaDB escuchará en todas las interfaces.

Será necesario reiniciar el servicio:

```
sudo systemctl restart mariadb
```

A continuación ejecutaré el script que tiene MariaDB para configurar el servicio:

```
sudo mariadb-secure-installation
```

Por último crearé un usuario y una base de datos:

```
sudo mariadb -u root -p
CREATE USER 'usuario'@'192.168.122.%' IDENTIFIED BY 'usuario';
CREATE DATABASE db;
GRANT ALL PRIVILEGES ON db.* TO 'usuario'@'192.168.122.%;
```

Nota: El usuario se debe de crear indicando la dirección de red para que solamente se pueda conectar a el los equipos de esa red.

Cliente

Lo primero que haré en el cliente será instalar el paquete para poder conectarme a la base de datos:

```
sudo apt install mariadb-client -y
```

Ahora por último para conectarme a la base de datos usaré el siguiente comando:

```
mariadb -h 192.168.122.126 -u usuario -D db -p
```

Instalación y configuración de MongoDB

Servidor

Para instalar el servidor de MongoDB por defecto apt no tiene candidato para instalación pero MongoDB da unos [archivos](#) .deb para instalarlo.

Lo primero que haré será descargar el fichero .deb e instalarlo:

```
wget https://repo.mongodb.org/apt/debian/dists/bookworm/mongodb-org/8.2/main/binary-amd64/mongodb-org-server_8.2.1_amd64.deb
sudo dpkg -i mongodb-org-server_8.2.1_amd64.deb
sudo apt install -f -y
```

Para establecer conexiones remotas en MongoDB se debe modificar el siguiente fichero:

```
sudo nano /etc/mongod.conf
```

Y se debe modificar la siguiente línea:

```
bindIp: 0.0.0.0
```

En ese mismo fichero para habilitar la autenticación se debe añadir lo siguiente:

```
security:
  authorization: "enabled"
```

MongoDB no permite rango de direcciones por lo que o se permite todas o solo se permite la localhost.

Nota: Con ayuda de iptables se puede modificar para que solo se permitan conexiones de la misma red.

Para aplicar los cambios hará falta reiniciar el servicio:

```
sudo systemctl restart mongod.service
```

Ahora voy a crear un usuario administrador:

```
mongosh
use admin
db.createUser({
  user: "admin",
  pwd: "admin",
  roles: [ { role: "root", db: "admin" } ]
})
```

Por último crearé un usuario y una base de datos:

```
mongosh -u admin
use db
db.createUser({
  user: "usuario",
  pwd: "usuario",
  roles: [ { role: "dbOwner", db: "db" } ]
})
```

Cliente

Para la instalación del cliente ocurre exactamente lo mismo, no hay candidato para la instalación con apt.

Lo primero que haré será instalar el .deb e instalarlo:

```
wget https://repo.mongodb.org/apt/debian/dists/bookworm/mongodb-org/8.2/main/binary-amd64/mongodb-mongosh\_2.5.8\_amd64.deb
sudo dpkg -i mongodb-mongosh_2.5.8_amd64.deb
```

Por último voy a realizar la conexión remota con el siguiente comando:

```
mongosh --host 192.168.122.187 -u usuario --authenticationDatabase db
```

Instalación y configuración de Apache Cassandra

Servidor

Lo primero para instalar el servidor Cassandra será instalar los requisitos previos, los cuales son tener java instalado, ya que Cassandra está escrita en Java.

En Debian 12 que es la versión la cual es más recomendable instalar Cassandra por problemas con la versión de python3, por defecto la versión de Java que se puede instalar es la 17 pero Cassandra solamente funciona con la 11 por lo que para instalarla me descargaré el .deb de este [enlace](#):

```
sudo dpkg -i jdk-11.0.28_linux-x64_bin.deb
```

Una vez instaladas las dependencias empezaré con la instalación de Cassandra como la indica la [guía](#):

```
sudo curl -o /etc/apt/keyrings/apache-cassandra.asc  
https://downloads.apache.org/cassandra/KEYS  
echo "deb [signed-by=/etc/apt/keyrings/apache-cassandra.asc]  
https://deb.debian.org/debian/cassandra 41x main" | sudo tee -a  
/etc/apt/sources.list.d/cassandra.sources.list  
sudo apt update  
sudo apt install cassandra -y
```

A continuación voy a configurar Cassandra para que acepte conexiones remotas, para ello modificaré el fichero de configuración **/etc/cassandra/cassandra.yaml**:

```
listen_address: 192.168.122.229    # Para que Cassandra escuche no solo en  
localhost  
rpc_address: 0.0.0.0              # Para aceptar conexiones de clientes  
broadcast_rpc_address: 192.168.122.229    # Para indicar al cliente la IP  
del servidor
```

También cambiaré en el mismo fichero las siguientes líneas:

```
authenticator: PasswordAuthenticator    # Activa la autenticación con  
usuario y contraseña  
authorizer: CassandraAuthorizer         # Activa el sistema de permisos y roles
```

Nota: Si la configuración de authorizer estuviera en AllowAllAuthorizer que es como está por defecto cualquier usuario puede hacer cualquier cosa.

Y la última modificación que haré por ahora en el archivo es la siguiente:

```
seed_provider:  
  - class_name: org.apache.cassandra.locator.SimpleSeedProvider  
    parameters:  
      - seeds: "192.168.122.229"
```

Nota: Esta última configuración indica los nodos de arranque de Cassandra, en un cluster real con varios nodos se indica la dirección IP de los diferentes equipos separadas por coma, como en mi caso solo hay una máquina bastará con poner la dirección IP del equipo.

Para aplicar los cambios se debe reiniciar el servicio:

```
sudo systemctl restart cassandra.service
```

Ahora entraré y crearé un keyspace (base de datos) y un usuario:

```
cqlsh -u cassandra -p cassandra
CREATE ROLE usuario WITH PASSWORD = 'usuario' AND LOGIN = true;
CREATE KEYSPACE db WITH replication = {'class': 'SimpleStrategy',
'replication_factor' : 1};
GRANT ALL PERMISSIONS ON KEYSPACE db TO usuario;
```

En Cassandra los usuarios se crean como roles, entonces si es un usuario se le añade la línea '*LOGIN = true*' porque si no se indica será un rol de permisos (como un grupo en Linux), es útil para agrupar permisos y heredarlos.

A los keyspaces de Cassandra (base de datos) hay que definirles un replication ya que este sistema está diseñado para ser distribuido y tolerante a fallos, es decir que no se use en un solo nodo para la base de datos. Teniendo esto en cuenta con la línea '*WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};*' se indica lo siguiente:

- **SimpleStrategy** → Es una estrategia de replicación, esta se suele usar para entornos de prueba o para entornos con un solo equipo para la base de datos. Para entornos de producción se usa *NetworkTopologyStrategy*.
- **replication_factor** → Esto indica el número de copias de cada dato, en mi caso 1, porque solamente uso un único nodo para Cassandra.

Ciente

Para instalar el cliente lo primero será cargar las claves y el repositorio de Cassandra al igual que en el servidor:

```
sudo curl -o /etc/apt/keyrings/apache-cassandra.asc
https://downloads.apache.org/cassandra/KEYS
echo "deb [signed-by=/etc/apt/keyrings/apache-cassandra.asc]
https://debian.cassandra.apache.org 41x main" | sudo tee -a
/etc/apt/sources.list.d/cassandra.sources.list
sudo apt update
```

Nota: Al igual que el servidor, el cliente de Cassandra es recomendable que sea instalado en Debian 12 como mucho, ya que en Debian 13 la versión de python3 da problemas.

Después ya podremos instalar el paquete `cassandra-tools` que es el que contiene las herramientas, como por ejemplo el binario para conectarse remotamente (`cqlsh`):

```
sudo apt install cassandra-tools -y
```

Por último me conectaré remotamente con el siguiente comando:

```
cqlsh 192.168.122.229 -u usuario
```

Uso básico de Cassandra

Ahora voy a explicar un poco como se usa Cassandra.

Cassandra no usa tablas como en los sistemas relacionales aunque usa comandos parecidos a los sistemas relacionales.

1. Crear keyspace (base de datos)

```
CREATE KEYSPACE tienda WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 1};
```

`SimpleStrategy` se usa en entornos de prueba. En producción se usa `NetworkTopologyStrategy`.

2. Usar el keyspace

```
USE tienda;
```

3. Crear una tabla

```
CREATE TABLE productos (
    id UUID PRIMARY KEY,
    nombre text,
    categoria text,
    precio decimal
);
```


4. Insertar datos

```
INSERT INTO productos (id, nombre, categoria, precio)
VALUES (uuid(), 'Laptop Lenovo', 'Electrónica', 2500.00);

INSERT INTO productos (id, nombre, categoria, precio)
VALUES (uuid(), 'Teclado Mecánico', 'Accesorios', 120.50);
```

5. Consultar datos

```
SELECT * FROM productos;
```

Cassandra devuelve todas las filas de la tabla.

Importante: solo puedes filtrar por columnas que estén en la clave primaria o tengan índices secundarios.

6. Consultas con filtro básico

Si la tabla tiene una clave primaria id, puedes consultar así:

```
SELECT nombre, precio FROM productos WHERE id = <ID_DEL_PRODUCTO>;
```

Si necesitas filtrar por otra columna (por ejemplo, categoria), primero debes crear un índice:

```
CREATE INDEX ON productos (categoria);

SELECT * FROM productos WHERE categoria = 'Electrónica';
```

7. Actualizar datos

```
UPDATE productos
SET precio = 2300.00
WHERE id = <ID_DEL_PRODUCTO>;
```

8. Borrar datos

```
DELETE FROM productos WHERE id = <ID_DEL_PRODUCTO>;
```

9. Ver estructuras y metadatos

```
DESCRIBE KEYSPACES;      -- Muestra todos los keyspaces
DESCRIBE TABLES;        -- Muestra tablas del keyspace actual
DESCRIBE TABLE productos; -- Muestra estructura detallada
```

10. Borrar tabla o keyspace

```
DROP TABLE productos;
DROP KEYSPACE tienda;
```

Creación aplicación web

Lo primero que haré será identificar las tecnologías que usaré para lograr el objetivo, en mi caso usaré Python3 + Flask (Jinja2).

1. Empezaré creando un entorno virtual python para aislar el proyecto y hacerlo portable:

```
> mkdir web-application
> cd web-application
> python3 -m venv venv
> source venv/bin/activate
```

2. A continuación instalaré las primeras dependencias y las añadiré a un archivo para hacer más portable el proyecto:

```
> pip install flask pymongo
> pip freeze > requirements.txt
```

Nota: Para instalar las dependencias en un futuro se hará con el comando:
pip install -r requirements.txt

3. Ahora voy a construir la estructura base del proyecto:

```
> mkdir -p templates static/css
> touch app.py templates/{base.html,index.html} static/css/style.css
```

4. Por último crearé el [código](#) de la aplicación web completa y ejecutaré el servidor en local:

```
flask run --debug
```

[Inicio](#) [Login](#)

Aplicación Web conectada a MongoDB

Bienvenido. Por favor, inicia sesión:

Usuario:

Contraseña:

Iniciar sesión

[Inicio](#) [Login](#)

Aplicación Web conectada a MongoDB

Bienvenido, usuario

Colecciones disponibles:

- [ventas](#)
- [productos](#)
- [clientes](#)

[Cerrar sesión](#)

[Inicio](#) [Login](#)

Aplicación Web conectada a MongoDB

Bienvenido, usuario

Colecciones disponibles:

- [ventas](#)
- [productos](#)
- [clientes](#)

Documentos en la colección: ventas

- {
 "_id": 1,
 "cantidad": 2,
 "cliente_id": 12,
 "fecha": "Sun, 05 Oct 2025 10:20:00 GMT",
 "producto_id": 3
}
- {
 "_id": 2,
 "cantidad": 1,
 "cliente_id": 25,
 "fecha": "Fri, 03 Oct 2025 14:50:00 GMT",
 "producto_id": 14
}
- {
 "_id": 3,
 "cantidad": 4,
 "cliente_id": 7,
 "fecha": "Tue, 07 Oct 2025 18:15:00 GMT",
 "producto_id": 9
}

Parte grupal.

Neo4j.

Introducción.

Neo4j es una base de datos de "grafos". Un grafo es como un mapa de conexiones, no una tabla. Imagina dibujar:

- Círculos (nodos): Representan cosas, como "Ana", "Juan" o "Madrid".
- Líneas (relaciones): Conectan los círculos, como "Ana ES_AMIGA_DE Juan" o "Juan VIVE_EN Barcelona".
- Notitas (propiedades): En cada círculo o línea, pones detalles extras, como "Edad: 25" en el nodo de Ana.

En lugar de buscar en una lista interminable, solo sigues las líneas para ver cómo todo está conectado. Esto hace que sea rápido y fácil encontrar patrones, incluso con millones de conexiones.

Se usa cuando los datos son conectados y enredados, como en la vida real. No para listas simples (como "ingredientes de una receta"), sino para cosas como:

- Redes sociales: "¿Qué amigos en común tengo con alguien?" (como en Facebook).
- Recomendaciones: "Si te gusta esta película, te gustará esta otra porque la vieron tus amigos" (como en Netflix o Amazon).
- Detección de fraudes: En bancos, ven si alguien está conectando cuentas falsas de forma sospechosa.
- Mapas y logística: "¿Cuál es la ruta más rápida entre ciudades, considerando tráfico y amigos que conozcas por el camino?"

Lo genial: Neo4j es fácil de cambiar. Si mañana quieres agregar "Ana ES_TIA_DE alguien", solo dibujas una línea nueva, sin rehacer toda la tabla.

Ejemplo:

En una tienda online (como Amazon): Imagina que compras un libro de Harry Potter. Neo4j conecta: Tú → Compraste → Libro → Lo leyó → Tu amigo → Quiere → Juguetes de magia. ¡Te recomienda un juguete en segundos! Empresas como eBay lo usan para sugerir productos y vender más.

En la NASA: Tienen datos de misiones antiguas. Neo4j conecta: Misión 1 → Usó → Pieza X → Falló en → Condición Y. Los científicos encuentran problemas rápido, como "esta pieza falló en Marte, ¿qué pasó en la Luna?", para no repetir errores.

Instalación bajo Debian13.

Neo4j necesita Java 21 para funcionar:

```
[0]marina@debian:~$ sudo apt install openjdk-21-jre-headless -y
```

Agregamos el repositorio oficial de Neo4j:

```
[0]marina@debian:~$ wget -O - https://debian.neo4j.com/neotechnology.gpg.key  
| sudo gpg --dearmor -o /etc/apt/keyrings/neotechnology.gpg
```

Agregamos la línea al repositorio:

```
[0]marina@debian:~$ echo 'deb  
[signed-by=/etc/apt/keyrings/neotechnology.gpg] https://debian.neo4j.com  
stable latest' | sudo tee -a /etc/apt/sources.list.d/neo4j.list
```

```
[0]marina@debian:~$ sudo apt update
```

Verificamos si está disponible:

```
[100]marina@debian:~$ apt list -a neo4j  
neo4j/stable 1:2025.09.0 all  
neo4j/stable 1:2025.08.0 all  
neo4j/stable 1:2025.07.1 all  
neo4j/stable 1:2025.07.0 all  
neo4j/stable 1:2025.06.2 all  
neo4j/stable 1:2025.06.1 all  
neo4j/stable 1:2025.06.0 all  
neo4j/stable 1:2025.05.1 all  
neo4j/stable 1:2025.05.0 all  
neo4j/stable 1:2025.04.0 all  
neo4j/stable 1:2025.03.0 all  
neo4j/stable 1:2025.02.0 all  
neo4j/stable 1:2025.01.0 all
```

Instalamos la última versión:

```
[0]marina@debian:~$ sudo apt install neo4j=1:2025.09.0 -y
```

Configuramos la contraseña:

```
[2]marina@debian:~$ sudo neo4j-admin dbms set-initial-password CONTRASEÑA
```

Iniciamos el servicio:

```
[0]marina@debian:~$ sudo systemctl start neo4j  
[0]marina@debian:~$ sudo systemctl enable neo4j
```

Comprobamos:

```
http://localhost:7474
```

Connect to Neo4j

Database access might require an authenticated connection

Connect URL

neo4j://localhost:7687

Authentication type

Username / Password

Username

neo4j

Password

.....

Connect

Connect to Neo4j

Database access might require an authenticated connection

New password

123marina

OR

Generate

Repeat new password

123marina

Change password

Para entrar desde la terminal:

```
[0]marina@debian:~$ cypher-shell -u neo4j -p 123marina
Connected to Neo4j using Bolt protocol version 5.8 at neo4j://localhost:7687
as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j>
```

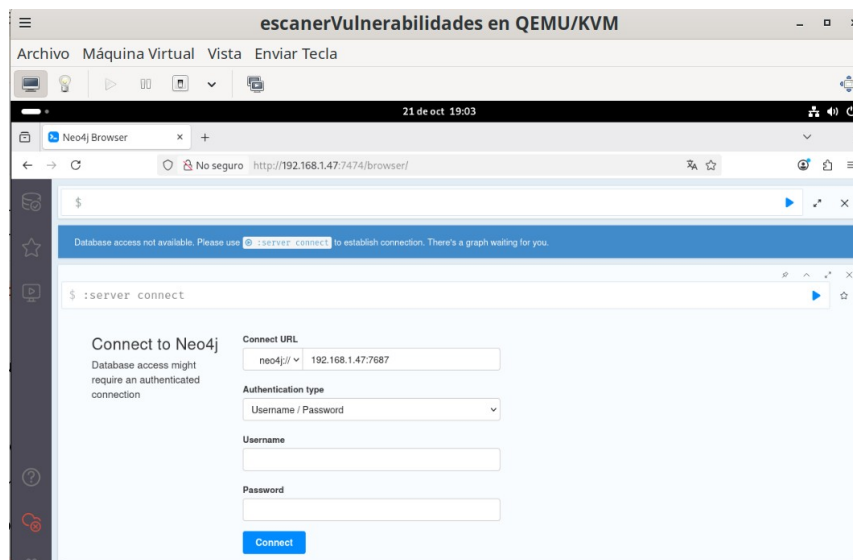
Conexión remota.

```
[0]marina@debian:~$ sudo nano /etc/neo4j/neo4j.conf  
server.default_listen_address=0.0.0.0
```

Reiniciamos el servicio y permitimos puertos:

```
[0]marina@debian:~$ sudo ufw allow 7474/tcp comment 'Neo4j HTTP Browser'  
Rules updated  
Rules updated (v6)  
[0]marina@debian:~$ sudo ufw allow 7687/tcp comment 'Neo4j Bolt'  
Rules updated  
Rules updated (v6)
```

Comprobamos:



Y desde la terminal, después de instalar neo4j:

```
cypher-shell -u neo4j -p 123marina -a bolt://192.168.1.47:7687
```

Comenzando desde la terminal.

Cypher Shell es la terminal de Neo4j, donde escribiremos las consultas Cypher directamente. Para ejecutar neo4j:

```
[0]marina@debian:~$ cypher-shell -u neo4j -p 123marina
```

Para una consulta básica:

```
neo4j@neo4j> RETURN "Hola Mundo" AS saludo;
+-----+
| saludo |
+-----+
| "Hola Mundo" |
+-----+

1 row
```

Crear datos (nodos y relaciones):

- Crear nodo (círculo) para una persona:

```
neo4j@neo4j> CREATE (ana:Persona {nombre: 'Ana', edad: 25});
```

```
neo4j@neo4j> CREATE (juan:Persona {nombre: 'Juan', edad: 30});
```

Conectamos a ana y a juan:

```
neo4j@neo4j> MATCH (a:Persona {nombre: 'Ana'}), (j:Persona {nombre: 'Juan'})
CREATE (a)-[:AMIGO_DE {desde: 2023}]->(j);
```

Consultar datos:

- Mostrar todas las personas:

```
neo4j@neo4j> MATCH (p:Persona) RETURN p.nombre, p.edad;
+-----+
| p.nombre | p.edad |
+-----+
| "Ana"    | 25     |
| "Juan"   | 30     |
+-----+

2 rows
```


- Conexiones:

```
neo4j@neo4j> MATCH (p:Persona) -[r:AMIGO_DE]->(amigo) RETURN p.nombre AS
persona, amigo.nombre AS amigo, r.desde AS año;
+-----+
| persona | amigo  | año   |
+-----+
| "Ana"   | "Juan" | 2023  |
+-----+

1 row
```

MATCH: Busca patrones + RETURN: devuelve resultados + opciones como CREATE o DELETE, etc.

Es como decir: Encuentra esto en el grafo y muéstrame aquello.

P:Persona → Es un nodo, donde p es como un alias y Persona es una etiqueta. Solo busca nodos marcados como Persona (Encuentra un nodo llamado 'p' que sea de tipo Persona).

-[r:AMIGO_DE]: Tipo de relación.

->: Dirección de la flecha. Significa "de p hacia amigo" (unidireccional). Si quieres bidireccional, usa -[]- (sin flechas).

Encuentra una conexión de tipo 'AMIGO_DE' que salga desde 'p' y vaya hacia otro nodo.

(amigo): Otro nodo.

Lo que hace todo el MATCH: Busca todas las personas (p) que tengan una relación 'AMIGO_DE' hacia otra persona (amigo).

RETURN: Devuelve los datos en una tabla.

p.nombre: toma el nombre del codo p.

AS persona: renombra el resultado.

Muestra una tabla con columnas: persona, amigo, año". Solo las propiedades pedidas, no el grafo entero (para no sobrecargar).

- Limpiar datos:

```
neo4j@neo4j> MATCH (p:Persona) DETACH DELETE p;
```

Lo borra todo.

Usando python.

Lo primero será instalar la librería de neo4j:

```
s/venv$ pip install neo4j
```

A continuación crearemos un código básico:

```
from neo4j import GraphDatabase

#conexión
URI = "bolt://localhost:7687" # Bolt es el protocolo de Neo4j
USERNAME = "neo4j"
PASSWORD = "123marina"

class Neo4jConnector:
    def __init__(self, uri, user, pwd):
        self.driver = GraphDatabase.driver(uri, auth=(user, pwd))

    def close(self):
        self.driver.close()

    def query(self, query, parameters=None, db_name=None):
        with self.driver.session(database=db_name) as session:
            result = session.run(query, parameters)
            return [record for record in result]

# Uso
connector = Neo4jConnector(URI, USERNAME, PASSWORD)

# Consulta simple
resultado = connector.query("RETURN 'Hola Mundo' AS saludo")
print(resultado[0]['saludo'])

connector.close()
```

Esto nos devuelve como resultado:

```
Hola Mundo
```

Crear y consultar datos:

```
from neo4j import GraphDatabase

#conexión
URI = "bolt://localhost:7687" # Bolt es el protocolo de Neo4j
USERNAME = "neo4j"
PASSWORD = "123marina"

class Neo4jConnector:
    def __init__(self, uri, user, pwd):
        self.driver = GraphDatabase.driver(uri, auth=(user, pwd))

    def close(self):
        self.driver.close()

    def query(self, query, parameters=None, db_name=None):
        with self.driver.session(database=db_name) as session:
            result = session.run(query, parameters)
            return [record for record in result]

# Uso
connector = Neo4jConnector(URI, USERNAME, PASSWORD)

# Crea nodos
connector.query("""
CREATE (ana:Persona {nombre: 'Ana', edad: 25})
""")

connector.query("""
CREATE (juan:Persona {nombre: 'Juan', edad: 30})
""")

# Crea relación
connector.query("""
MATCH (a:Persona {nombre: 'Ana'}), (j:Persona {nombre: 'Juan'})
CREATE (a)-[:AMIGO_DE {desde: 2023}]->(j)
""")

# Consulta
amigos = connector.query("""
MATCH (p:Persona)-[r:AMIGO_DE]->(amigo)
RETURN p.nombre AS persona, amigo.nombre AS amigo, r.desde AS año
""")

for amigo in amigos:
    print(f"{amigo['persona']} es amigo de {amigo['amigo']} desde {amigo['año']}")

# Limpia (opcional)
connector.query("MATCH (p:Persona) DETACH DELETE p")

connector.close()
```

Esto nos devuelve:

```
Ana es amigo de Juan desde 2023
```

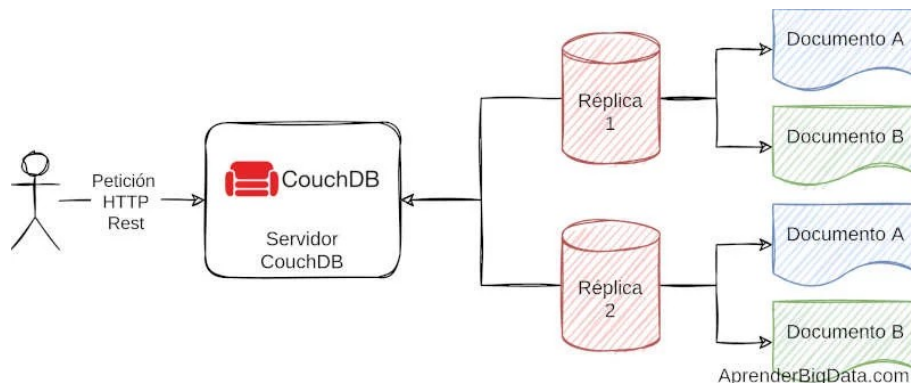
CouchDB.

Apache CouchDB es una base de datos de documentos NoSQL de código abierto que recopila y almacena datos en formatos de documentos basados en JSON. A diferencia de las bases de datos relacionales, CouchDB utiliza un modelo de datos sin esquemas, lo que simplifica la gestión de registros en varios dispositivos informáticos, teléfonos móviles y navegadores web.

CouchDB es de código abierto, de uso gratuito y se integra fácilmente en su infraestructura actual de gestión de datos. Debido a que tiene más control sobre el software, también tiene más flexibilidad a la hora de adaptarlo a las necesidades únicas de su negocio. Tanto si necesita un almacén de documentos de uso general, como si desea una sincronización de datos eficaz o adoptar una mentalidad "offline first", CouchDB ofrece a las empresas la flexibilidad que necesitan para crear infraestructuras duraderas, fiables y escalables.

CouchDB viene con un conjunto de características diseñadas para reducir el esfuerzo que supone ejecutar un sistema distribuido resistente.

Está orientada a documentos con replicación bidireccional, vistas dinámicas con MapReduce, y API HTTP/REST. Soporta trabajo fuera de línea, almacenamiento de documentos JSON y binarios, y es compatible y flexible para múltiples sistemas, lenguajes y entornos distribuidos.



Instalación.

He tenido varios problemas con la instalación de CouchDB en debian13, ya que todavía no hay una versión para este. Voy a realizar la instalación con Docker, ya que aislamos CouchDB en un contenedor, sin mezclar paquetes antiguos con mi sistema debian13.

Lo primero que haremos será añadir repositorios seguros y usar HTTPS:

```
[0]marina@debian:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
```

Añadiremos la clave GPG de Docker, ya que Docker firma sus paquetes y necesitamos confiar en ellos:

```
[0]marina@debian:~$ sudo mkdir -p /etc/apt/keyrings
[0]marina@debian:~$ curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Esto crea un archivo con la clave GPG en /etc/apt/keyrings/docker.gpg.

Añadimos el repositorio oficial de Docker:

```
[0]marina@debian:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Instalamos Docker:

```
[100]marina@debian:~$ sudo apt install -y docker.io
```

Este paquete es mantenido por Debian y funciona en trixie. Habilitamos y arrancamos Docker:

```
[0]marina@debian:~$ sudo systemctl enable docker --now
```

Esto arranca Docker y lo configura para que se inicie automáticamente al encender la máquina.

Verificamos:

```
[0]marina@debian:~$ sudo docker run hello-world
```

Levantamos CouchDB indicando el bind address para conexión remota:

```
[0]marina@debian:~$ sudo mkdir -p ~/couchdb_data
[0]marina@debian:~$ sudo chown -R 998:998 ~/couchdb_data
#levantamos con usuario admin y persistencia:
[0]marina@debian:~$ sudo docker run -d --name couchdb -p 5984:5984 \
  -v ~/couchdb_data:/opt/couchdb/data \
  -e COUCHDB_USER=admin -e COUCHDB_PASSWORD=adminpassword \
  -e COUCHDB_BIND_ADDRESS=0.0.0.0 \
  couchdb:latest
287d3a8a0be09a66baa927ebb031d19952cfbc241a718c6cbbbf84b484e222e7
```

Verificamos que está corriendo:

```
[0]marina@debian:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS
NAMES
287d3a8a0be0   couchdb:latest "tini -- /docker-ent..." 26 seconds ago Up
25 seconds    4369/tcp, 9100/tcp, 0.0.0.0:5984->5984/tcp, :::5984->5984/tcp
couchdb
```

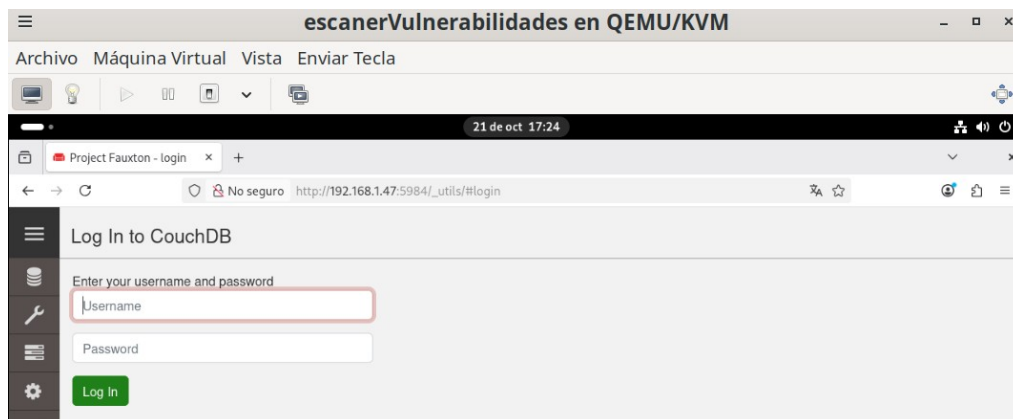
Para acceder desde el localhost:

```
http://localhost:5984/_utils
```

Para acceder desde otra máquina de la red:

```
http://192.168.1.47:5984/_utils
```

Comprobamos acceso remoto:



Creación de base de datos.

Para poder trabajar con python voy a crear un entorno virtual donde instalaré la librería de couchdb:

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$  
python3 -m venv venv  
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$  
source venv/bin/activate
```

Un ejemplo en código python sería:

```
#Importar la libreria CouchDB  
import couchdb  
  
#conexión al servidor  
couch = couchdb.Server('http://admin:adminpassword@localhost:5984/')  
  
#crear base de datos  
db = couch.create('prueba')  
  
#documento en la base  
doc = {'name': 'John Doe', 'age': 34}  
db.save(doc)  
  
#Recuperar documento  
doc = db.get('prueba')  
  
#mostrar documento  
print(doc)
```

Comprobamos:



Name	Size	# of Docs	Partitioned	Actions
prueba	332 bytes	1	No	  

Si quisiéramos comprobarlo desde la terminal:

Listar bases de datos.

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$ curl  
-X GET http://admin:adminpassword@localhost:5984/_all_dbs  
["prueba"]
```

CouchDB es un servidor HTTP, por lo que debemos acceder a los datos con herramientas como curl.

Ver los documentos de la base.

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$ curl
-X GET http://admin:adminpassword@localhost:5984/prueba/_all_docs
{"total_rows":1,"offset":0,"rows":[
{"id":"66ae699eb61582029d2dd2f0df001b2b","key":"66ae699eb61582029d2dd2f0df00
1b2b","value":{"rev":"1-b12f81e523807c1a2ddcad4ba7f97487"}}
]}
```

Esto devuelve los IDs y revisiones de todos los documentos:

```
1 {
2   "_id": "66ae699eb61582029d2dd2f0df001b2b",
3   "_rev": "1-b12f81e523807c1a2ddcad4ba7f97487",
4   "name": "John Doe",
5   "age": 34
6 }
```

Ver documento específico.

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$ curl
-X GET
http://admin:adminpassword@localhost:5984/prueba/66ae699eb61582029d2dd2f0df0
01b2b
{"_id":"66ae699eb61582029d2dd2f0df001b2b","_rev":"1-
b12f81e523807c1a2ddcad4ba7f97487","name":"John Doe","age":34}
```

Esto nos devuelve el documento completo del JSON.

Ver documentos con contenido completo.

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$ curl
-X GET http://admin:adminpassword@localhost:5984/prueba/_all_docs?
include_docs=true
{"total_rows":1,"offset":0,"rows":[
{"id":"66ae699eb61582029d2dd2f0df001b2b","key":"66ae699eb61582029d2dd2f0df00
1b2b","value":{"rev":"1-b12f81e523807c1a2ddcad4ba7f97487"},"doc":
{"_id":"66ae699eb61582029d2dd2f0df001b2b","_rev":"1-
b12f81e523807c1a2ddcad4ba7f97487","name":"John Doe","age":34}}
]}
```

Esto devuelve todos los documentos completos, no solo sus IDs.

Guardar documento desde la terminal.

```
curl -X POST http://admin:adminpassword@localhost:5984/prueba \
-H "Content-Type: application/json" \
-d '{"name": "Marina", "age": 24}'
```

Esto crea un nuevo documento y devuelve el id y rev. Para comprobar:

```
[0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-clientes$ curl
-X GET
http://admin:adminpassword@localhost:5984/prueba/66ae699eb61582029d2dd2f0df0
04635
{"_id": "66ae699eb61582029d2dd2f0df004635", "_rev": "1-
78288dddfa27b168783b83c839fc8166", "name": "Marina", "age": 24}
```

Usando python.

```
#Importar la libreria CouchDB
import couchdb

db_name = 'prueba'

#conexión al servidor
couch = couchdb.Server('http://admin:adminpassword@localhost:5984/')

#crear base de datos y coenctarnos
if db_name in couch:
    db = couch[db_name]
    print(f"Base de datos '{db_name}' ya existe.")
else:
    db = couch.create(db_name)
    print(f"Base de datos '{db_name}' creada")

#documento en la base
doc = {'name': 'Elena', 'age': 23}
doc_id, doc_rev = db.save(doc)
print(f"Documento guardado con id: {doc_id}")

#Recuperar documento
doc = db.get(doc_id)
print("Documento recuperado: ")
print(doc)

#listar todos los documentos
print("\nTodos los documentos en la base: ")
for doc_id in db:
    print(db[doc_id])
```

Salida:

```
(venv) [0]marina@debian:~/Escritorio/BBDD/tema1/practicas/servidores-
clientes/venv$ /home/marina/Escritorio/BBDD/tema1/practicas/servidores-
clientes/venv/bin/python
/home/marina/Escritorio/BBDD/tema1/practicas/servidores-clientes/venv/
couch.py
Base de datos 'prueba' ya existe.
Documento guardado con id: 66ae699eb61582029d2dd2f0df004803
Documento recuperado:
<Document '66ae699eb61582029d2dd2f0df004803'@'1-
2df8e065fb4e395aeaca28e70b291707' {'name': 'Elena', 'age': 23}>

Todos los documentos en la base:
<Document '66ae699eb61582029d2dd2f0df001b2b'@'1-
b12f81e523807c1a2ddcad4ba7f97487' {'name': 'John Doe', 'age': 34}>
<Document '66ae699eb61582029d2dd2f0df004635'@'1-
78288dddfa27b168783b83c839fc8166' {'name': 'Marina', 'age': 24}>
<Document '66ae699eb61582029d2dd2f0df004803'@'1-
2df8e065fb4e395aeaca28e70b291707' {'name': 'Elena', 'age': 23}>
```

Conclusión.

CouchDB es útil cuando necesitas:

- Aplicaciones web o móviles con sincronización offline
 - Por ejemplo, apps que deben funcionar sin internet y luego sincronizar cambios con el servidor.
 - CouchDB tiene replicación y resolución de conflictos integrada.
- Datos semi-estructurados o cambiantes
 - Como JSON, puedes agregar o quitar campos sin afectar otros documentos.
 - Ejemplo: usuarios con distintos atributos según su tipo.
- Sistemas distribuidos
 - Permite replicar bases de datos entre servidores y manejar conflictos automáticamente.
- APIs REST directamente
 - Puedes exponer tus datos sin necesidad de crear un backend complejo.

Memcached.

Explicación del SGBD

Memcached es un sistema de almacenamiento en caché en memoria (RAM) de:

- **Clave-Valor** → Estructura simple de datos (clave: valor).
- **Distribuido** → Usa varios servidores como una sola caché.
- **Alto rendimiento** → Operaciones en memoria, rápidas y ligeras.

El proposito de este SGBD es acelerar aplicaciones web, reduciendo la carga en bases de datos o servicios externos, guardando temporalmente datos ya consultados o procesados.

Su funcionamiento es el siguiente:

- Los datos se guardan como pares: clave → valor.
- Se almacenan en memoria RAM, lo que permite accesos muy rápidos.
- Cuando la memoria se llena, reemplaza los datos menos usados (algoritmo LRU).

Algunas características claves son:

- Muy rápido y ligero (escrito en C).
- Distribuido → se unen varios servidores Memcached como un solo pool de caché.
- No persistente → si se reinicia, se pierden los datos.
- Es usado por grandes plataformas como: Facebook, Twitter, Wikipedia, etc.

Como es un SGBD que no guarda sus datos permanentemente su uso es diferente al de otros tipos:

- Cachear resultados de consultas SQL
- Guardar sesiones de usuario
- Cachear resultados de APIs o plantillas renderizadas.

Memcached no usa cliente como por ejemplo PostgreSQL, si no que usa bibliotecas para que memcached se pueda comunicar con diferentes lenguajes de programación como por ejemplo:

- **Python** → pylibmc, python-memcached, pymemcache
- **PHP** → Memcached, Memcache
- **Node.js** → memjs, memcached
- **Java** → spymemcached, XMemcached
- **Go** → gomemcache
- **Ruby** → dalli

Los nombrados anteriormente no son clientes CLI de Memcached, si no que son librerías / extensiones que usan diferentes lenguajes para comunicarse.

Para pruebas se suele usar telnet, que es una herramienta que permite conectarse a otros equipos a través de la red usando una conexión de texto simple. Telnet te permite "hablar" directamente con un servidor por medio de comandos escritos, como si se abriera una sesión manualmente.

Por ejemplo, para memcached sería:

```
telnet <ip> 11211
```

Este comando conecta con un servidor memcached (default port 11211) y cuando se conecta se pueden escribir comandos del protocolo memcached, por ejemplo:

```
set saludo 0 60 5  
hola!  
get saludo
```

Y con esa entrada el servidor responde:

```
VALUE saludo 0 5  
hola!  
END
```

Más tarde en el apartado de uso de Memcached explicaré los comandos y que es 0 60 5 en los comandos anteriores.

Instalación y configuración para uso remoto

Para instalar Memcached es muy fácil ya que tiene candidato de instalación en Debian13 con apt:

```
sudo apt install memcached -y
```

Ahora pasaré a configurar el SGBD para su uso remoto. Esto es muy sencillo ya que será necesario modificar en el fichero `/etc/memcached.conf` la línea:

```
-l 127.0.0.1
```

por

```
-l 0.0.0.0
```

Después de modificar esto se reiniciará el servicio:

```
sudo systemctl restart memcached.service
```

Y con esta configuración ya se podrá acceder al servidor memcached remotamente, ya que escuchará cualquier IP y no solo la local (127.0.0.1).

Nota: Se puede comprobar con el comando `ss -tln`.

Para realizar la prueba yo usaré el siguiente comando:

```
telnet memcached 11211
```

Nota: En el anterior comando pongo memcached y no la IP, por que lo tengo configurado en el fichero de direcciones estáticas `/etc/hosts` para que el nombre apunte a la IP.

Aclarar que memcached en aplicaciones pequeñas y medianas se suele usar en el mismo servidor ya que así tendrá 0 latencia y configuraciones más simples. Pero en aplicaciones más grandes o distribuidas si se usa en servidores diferentes a los que se tienen que conectar las aplicaciones, esto lo hace más escalable y hace que no se consuma memoria del servidor de la aplicación.

Y otra aclaración es que memcached no tiene autenticación ni cifrado por lo que nunca debería de estar expuesto a Internet directamente, se debería de usar en una red interna entre los distintos servidores que lo usen.

Uso básico y explicación del funcionamiento

Funcionamiento de Memcached

Como ya expliqué antes, memcached es un sistema de caché en memoria, guarda datos temporalmente en la memoria RAM para evitar recalcular o consultar la base de datos cada vez.

El flujo básico es el siguiente:

- La aplicación pide un dato como por ejemplo el perfil del usuario 123.
- Primero busca en memcached.
- Si el dato está en memcached, lo devuelve rápido porque está en la memoria RAM.
- Si no está, la aplicación lo busca en la base de datos, lo guarda en memcached para la próxima vez, y lo devuelve.

A este proceso se le llama cache-aside pattern (leer de la caché, o llenar si no existe).

Lo que almacena memcached son pares clave → valor como por ejemplo:

```
"user:123" → "{ 'nombre': 'Ana', 'edad': 25 }"  
"pagina:home" → "<HTML del home>"  
"producto:456" → "{ 'precio': 99.99 }"
```

Todo esto lo guarda en la memoria RAM, no en el disco, y cuando se llena expulsa los datos menos usados usando el algoritmo LRU (Least Recently Used).

Uso básico

Ahora voy a explicar como se usa con comandos y con código (Python).

Desde Telnet (modo texto)

Para entrar telnet <IP> 11211:

```
telnet memcached 11211
```

Guardar un valor:

```
set saludo 0 60 5  
hola!
```

Esto guarda el valor "hola!" con clave "saludo"

- 0 → flags (normalmente 0)
- 60 → tiempo de expiración (segundos)
- 5 → longitud del valor

El servidor responde:

```
STORED
```

Nota: Las flags se suelen usar para marcar el tipo de dato por ejemplo: 0 sin definir, 1 json, 2 binario, 3 comprimido. Estos datos son para el programador de la aplicación, ya que pueden significar lo que decida en la aplicación.

Leer un valor:

```
get saludo
```

El servidor responde con:

```
VALUE saludo 0 5  
hola!  
END
```

Borrar un valor:

```
delete saludo
```

El servidor responde con:

```
DELETED
```

Desde un lenguaje (Python)

Lo primero que haré será crear un entorno virtual e instalar la librería para usar memcached en un programa Python:

```
python3 -m venv venv
source venv/bin/activate
pip install pymemcache
pip freeze > requirements.txt
```

El programa creado es el siguiente:

```
from pymemcache.client import base
# Conexión al servidor Memcached
client = base.Client(('192.168.122.14', 11211))
# Guardar un valor
client.set('usuario:1', 'Ana')
# Leerlo
valor = client.get('usuario:1')
print(valor)
# Borrarlo
client.delete('usuario:1')
```

Cuando se ejecuta devuelve lo siguiente:

```
b'Ana'
```

Nota: la b significa que el dato es de tipo byte.

En el programa python no se indica el TTL que es la vida útil del dato introducido, por lo que se mantendrá ahí indefinidamente hasta que se necesite eliminar.

Como memcached tiene una cantidad fija de RAM, cuando esta se llena es cuando se eliminan los datos menos usados recientemente.

Para indicar una flag o un TTL a un dato se haría de la siguiente manera:

```
client.set('clave', 'valor', expire=30, flags=0)
```


Vídeos

A continuación dejaremos los enlaces a los dos vídeos realizados para la práctica:

- [Herramienta de Administración web para MongoDB](#)
- [Videocomparativa de todos los SGBD NoSQL](#)